

Version 1.2

August 2026



MACHINE LEARNING

MODULE 4 - DEALING WITH OVERFITTING AND UNDERFITTING

Authored by:

Ir. Yufis Azhar, S.Kom., M.Kom.

Haidar Dimas Heryanto

Muhammad Aunul Hakim

INFORMATICS LABORATORY

UNIVERSITY OF MUHAMMADIYAH MALANG

TABLE OF CONTENTS

TABLE OF CONTENTS.....	2
INTRODUCTION.....	3
OBJECTIVES.....	3
MODULE OBJECTIVES.....	3
PREPARATION.....	3
MAIN TOPICS.....	4
A. Overfitting.....	4
B. Underfitting.....	5
C. Methods for Addressing Overfitting and Underfitting.....	7
1. Dealing with Underfitting.....	7
a. Data Tabular.....	7
b. Data Teks/Citra.....	11
2. Dealing with Overfitting.....	14
a. Overfitting Text and Image Data Using the Most Common Methods.....	14
1. REGULARIZATION.....	14
2. DROPOUT.....	14
3. EARLY STOPPING.....	15
4. FINE-TUNING MODEL.....	16
b. Overfitting in Image Data.....	17
DATA AUGMENTATION.....	17
c. Overfitting in Text Data.....	18
1. EMBEDDING DROPOUT.....	18
2. RANDOM SYNONYM REPLACEMENT.....	18
3. RANDOM INSERTION.....	19
4. RANDOM SWAP.....	20
5. RANDOM DELETION.....	20
CODELAB (10%).....	20
LAB ASSIGNMENT (90%).....	24
Lab Assignment 1.....	24
Lab Assignment 2.....	28



INTRODUCTION

OBJECTIVES

1. Students understand overfitting and underfitting in machine learning.
2. Students can identify main causes of overfitting and underfitting in machine learning.
3. Students understand various methods for addressing overfitting, including regularization, dropout, early stopping, fine-tuning, and data augmentation.
4. Students learn methods to fix underfitting, such as increasing model complexity, feature engineering, and longer training.
5. Students can distinguish handling strategies by data type (tabular, text, images).
6. Students can implement and evaluate strategies for handling overfitting and underfitting using Python, TensorFlow, and Keras.

MODULE OBJECTIVES

1. Students can identify signs of overfitting and underfitting in models through model evaluation results.
2. Students can apply data preprocessing, model tuning, and data augmentation to improve model performance.
3. Students can run experiments on different data types (tabular, text, images) to address fitting issues.
4. Students can produce models that are more general (not overfitted) and powerful (not underfitted), with more stable accuracy on both validation and test data.

PREPARATION

1. Muslim students may recite the following prayer before starting the lesson
رَضِيتُ بِاللّٰهِ رَبًّا وَبِالْإِسْلَامِ دِينًا وَبِمُحَمَّدٍ نَبِيًّا وَرَسُولًا. رَبِّ زِدْنِي عِلْمًا وَارْزُقْنِي فَهْمًا
2. Laptop/PC
3. Interactive Python Notebook (ipynb)
4. Python (version ≥ 3.10 recommended)
5. TensorFlow (version ≥ 2.10 recommended)
6. Keras, Matplotlib, seaborn, scikit-learn, pandas, numpy.
7. Source Code :

∞ MODULE 4 MACHINE LEARNING - DEALING WITH OVERFITTING AND UNDERFITTING

KEYWORDS

Overfitting, Underfitting, Regularization, Dropout, Early Stopping, Fine-tuning, Data Augmentation, Feature Engineering, CNN, Deep Learning



MAIN TOPICS

When building a machine learning model, high accuracy on training data is not always a good sign. A model that performs excellently during training can still fail when faced with new, unseen data. This chapter discusses two common problems that affect a model's ability to generalize—overfitting and underfitting—along with how to recognize and address them.

A. Overfitting

Overfitting occurs when a machine learning model is trained too closely on the training data, causing it to memorize patterns—even noise or irrelevant random details—to an excessive degree. As a result, the model loses its ability to generalize or make accurate predictions on new, previously unseen data (validation or test data). It's like a student who memorizes all the example problems and answers in a textbook without truly understanding the concepts. So, when presented with a slightly different problem, the student will struggle to answer it. Similarly, with overfitting, the model performs very well on data it's already familiar with (training data) but fails when presented with new data it has never seen before.

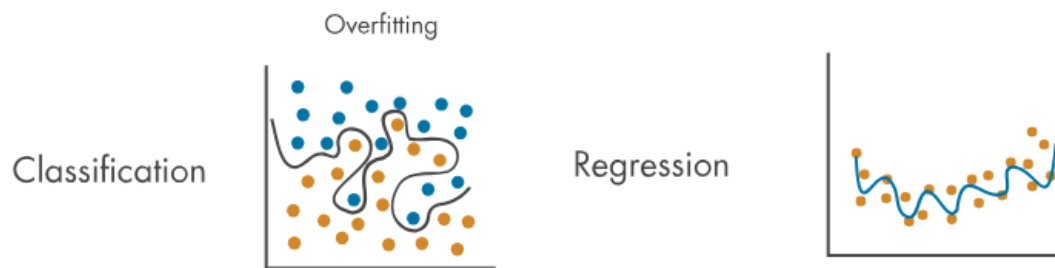


Figure 1. Illustration of Overfitting
Grafik Overfitting (source: mathworks.com)

The graph above shows an example of overfitting; the blue and orange points represent two different data categories. The winding black line is the decision boundary the model created. Notice how the winding black line in the classification graph encloses each blue and orange point—to the extent that some blue points are even surrounded. This indicates the model is too focused on the training data and will not generalize well if new data points shift slightly. Meanwhile, in the regression graph, the model creates a prediction line that is very jagged and sharply curved to “match” each training data point perfectly.

The main signs of overfitting are typically high performance on the training data compared to accuracy on the validation data. Training accuracy can reach 99% or 100%, while validation/test accuracy is significantly lower. Here are some causes of overfitting:



1. The model is too complex, with too many parameters or layers (for example, too many hidden layers), giving it too much “freedom” to memorize the data.
2. There is too little training data, so it cannot represent all possible patterns. As a result, the model learns only from the few available examples.
3. The training duration is too long, and the model is trained until it reaches an optimal state on the training data, past the point where it begins to lose its ability to generalize.

To address overfitting, we can try several common techniques:

1. **Cross-Validation**

The data is divided into several subsets. The model is then trained and validated alternately using different subsets. In this way, the model is thoroughly tested on diverse data, making the evaluation results more reliable.

2. **Regularization**

Apply a penalty to the model if it is too complex. The goal is to prevent the model from simply “memorizing” the data. Two commonly used techniques are L1 (Lasso) and L2 (Ridge) regularization.

3. **Dropout**

During training, some neurons are randomly selected to be temporarily “turned off.” This prevents the model from relying too heavily on specific neurons and helps improve generalization.

4. **Early Stopping**

The training process is monitored using validation loss. If validation loss begins to increase (a sign the model is starting to overfit), training stops automatically.

5. **Data Augmentation**

This technique increases the variety of training data by modifying existing data, for example, by rotating, flipping, or resizing images. The goal is to help the model learn from a more diverse dataset.

6. **Feature Selection**

The process of selecting truly relevant features and discarding irrelevant ones. This reduces complexity, minimizes noise, and makes the model more efficient.

B. Underfitting

Underfitting is the opposite of overfitting; it is a machine learning problem in which the model is too simple to capture the underlying patterns in the data. As a result, the model performs poorly on both the training data and new data. If overfitting is likened to a student who memorizes too many details, then underfitting is like a student who doesn’t study at all. The model fails to grasp the data’s fundamental concepts, so it fails at its task, whether on data it has seen before or data it has not.



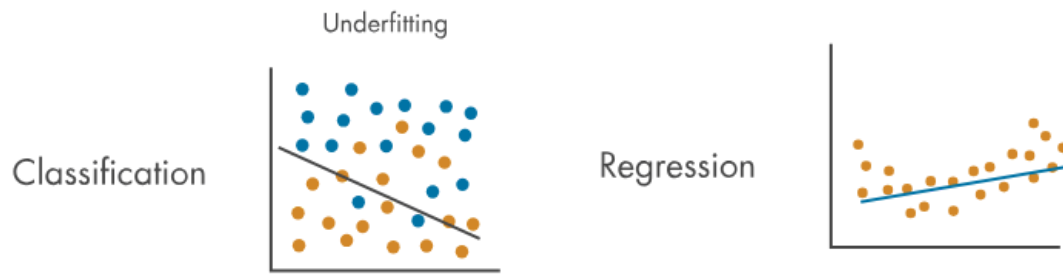


Figure 2. Illustration of Underfitting
Grafik *underfitting* (Source: [mathworks.com](https://www.mathworks.com))

The graph above is an example of underfitting. The blue and orange points represent two different data categories. The black straight line is the decision boundary the model creates. Note that the straight line in the classification graph fails to separate the blue and orange points properly. This indicates the model is too simple to capture the data patterns, resulting in many misclassified points.

Meanwhile, in the regression graph, the model produces a highly inaccurate prediction line—a straight line. The model fails to follow the data's winding pattern, as its prediction line cannot pass through most data points.

The main characteristics of an underfitted model are typically evident in its poor performance on both the training and validation data. The model performs poorly even on the data it was trained on.

Here are some causes of underfitting:

1. The model is too simple; it lacks enough parameters or layers to capture the data's complexity.
2. There is too little training data, or the data lacks relevant features to help the model make accurate decisions.
3. The training duration is too short; the model has not been trained long enough to learn from the available data.



To address overfitting, we can try several common techniques:

1. Increasing the Complexity of the Model

Underfitting usually occurs because the model is too simple. The solution is to use a more complex model architecture or increase the number of layers or neurons so that the model can better capture patterns in the data.

2. Reducing Regularization

Excessive regularization can make it difficult for the model to learn patterns. By reducing the regularization value (for example, the lambda parameter in L1/L2), the model is given more flexibility to learn.

3. Longer Training Sessions

Underfitting can occur if you stop training too early. By increasing the number of epochs, the model has more opportunities to learn patterns from the data.

4. Better Feature Engineering

Sometimes a model underfits because the features aren't informative enough. By adding new features, combining features, or changing how features are represented, the model can better understand the data.

5. Reducing Noise in Data

Too noisy data makes it hard to identify important patterns. Cleaning the data or performing the right preprocessing can help the model learn more accurately.

6. Using More Relevant Data

If the dataset is too small or not representative, the model will struggle to recognize patterns. Adding new, higher-quality, or more relevant data can reduce the risk of underfitting.

C. Methods for Addressing Overfitting and Underfitting

1. Dealing with Underfitting

To address underfitting, replace an overly simple model with a more complex one and adapt it to the data's needs. Here are some examples of how to address underfitting for different types of data.

a. Data Tabular

First, we'll create tabular data manually using NumPy.

```
# Buat data tabular sederhana
np.random.seed(0)
X = np.sort(5 * np.random.rand(80, 1), axis=0)
y = np.sin(X).ravel() + np.random.normal(0, 0.1, X.shape[0])
```



Next, split the data

```
# Split data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Now that the data is ready to be processed, let's try out a very simple model.

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
# Simple Neural Network (Underfitting)
simple_nn_model = Sequential([
    Dense(4, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(1)
])

simple_nn_model.compile(optimizer='adam', loss='mse')
simple_nn_history = simple_nn_model.fit(X_train, y_train, epochs=50,
batch_size=32, verbose=0)

print("Underfitting dengan Neural Network Sederhana:")
print(f"MSE Pelatihan: {simple_nn_model.evaluate(X_train, y_train,
verbose=0):.4f}")
print(f"MSE Pengujian: {simple_nn_model.evaluate(X_test, y_test,
verbose=0):.4f}\n")
```

The model above will produce the following MSE values:

```
Underfitting dengan Neural Network Sederhana:
MSE Pelatihan: 0.3582
MSE Pengujian: 0.5042
```



To address the problem of underfitting, let's try building a more complex

model:

```
# More Complex Neural Network (Addressing Underfitting)
complex_nn_model = Sequential([
    Dense(32, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(32, activation='relu'),
    Dense(1)
])

complex_nn_model.compile(optimizer='adam', loss='mse')
complex_nn_history = complex_nn_model.fit(X_train, y_train, epochs=50,
batch_size=32, verbose=0)

print("Mengatasi Underfitting dengan Neural Network yang Lebih Kompleks:")
print(f"MSE Pelatihan: {complex_nn_model.evaluate(X_train, y_train,
verbose=0):.4f}")
print(f"MSE Pengujian: {complex_nn_model.evaluate(X_test, y_test,
verbose=0):.4f}\n")
```

After training, the model will produce the following MSE values:

```
Mengatasi Underfitting dengan Neural Network yang Lebih Kompleks:
MSE Pelatihan: 0.1973
MSE Pengujian: 0.3143
```

By adding neurons and a few layers that are more complex than in the previous model, the MSE value is much lower than the MSE for the less complex model. Let's try increasing the number of epochs.

```
# More Complex Neural Network (Addressing Underfitting)
complex_nn_model = Sequential([
    Dense(32, activation='relu', input_shape=(X_train.shape[1],)),
    Dense(32, activation='relu'),
    Dense(1)
])

complex_nn_model.compile(optimizer='adam', loss='mse')
complex_nn_history = complex_nn_model.fit(X_train, y_train, epochs=100,
batch_size=32, verbose=0) #menambahkan epochs

print("Mengatasi Underfitting dengan Neural Network yang Lebih Kompleks:")
print(f"MSE Pelatihan: {complex_nn_model.evaluate(X_train, y_train,
verbose=0):.4f}")
```



```
print(f"MSE Pengujian: {complex_nn_model.evaluate(X_test, y_test,
verbose=0):.4f}\n")
```

The code above will produce the following MSE value:

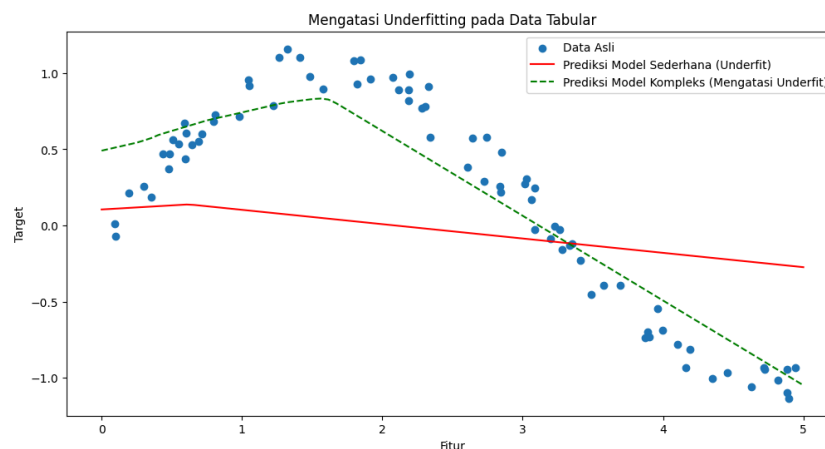
```
Mengatasi Underfitting dengan Neural Network yang Lebih Kompleks:
MSE Pelatihan: 0.0482
MSE Pengujian: 0.0904
```

The MSE value drops further if we add more training epochs. However, a more complex model doesn't necessarily mean better accuracy. If we go too far, we'll run into another problem: overfitting. Therefore, we must tune the model parameters until we achieve optimal performance. Use the following code to view a visualization of the model training results :

```
X_plot = np.linspace(0, 5, 100)[: , np.newaxis]
y_plot_simple = simple_nn_model.predict(X_plot)
y_plot_complex = complex_nn_model.predict(X_plot)

plt.figure(figsize=(12, 6))
plt.scatter(X, y, label='Data Asli')
plt.plot(X_plot, y_plot_simple, color='red', label='Prediksi Model Sederhana
(Underfit)')
plt.plot(X_plot, y_plot_complex, color='green', linestyle='--', label='Prediksi
Model Kompleks (Mengatasi Underfit)')
plt.title('Mengatasi Underfitting pada Data Tabular')
plt.xlabel('Fitur')
plt.ylabel('Target')
plt.legend()
plt.show()
```

The code above will produce the following visualization :



b. Data Teks/Citra

First, we'll use the MNIST data provided by Keras:

```
import numpy as np
import matplotlib.pyplot as plt
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Conv2D, Flatten, MaxPooling2D
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical

# Muat data MNIST
(X_train, y_train), (X_test, y_test) = mnist.load_data()
```

Next, perform data preprocessing

```
# Pre-processing data
X_train = X_train.reshape((60000, 28, 28, 1)).astype('float32') / 255
X_test = X_test.reshape((10000, 28, 28, 1)).astype('float32') / 255
y_train_cat = to_categorical(y_train)
y_test_cat = to_categorical(y_test)
```

Next, create a simple model.

```
simple_model_cnn = Sequential([
    Conv2D(16, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    Flatten(),
    Dense(10, activation='softmax')
])

simple_model_cnn.compile(optimizer='adam', loss='categorical_crossentropy',
metrics=['accuracy'])
simple_history = simple_model_cnn.fit(X_train, y_train_cat, epochs=1,
batch_size=64, validation_split=0.2)
print("\nUnderfitting dengan model sederhana (1 epoch):")
print(f"Akurasi pelatihan: {simple_history.history['accuracy'][-1]:.4f}")
print(f"Akurasi validasi: {simple_history.history['val_accuracy'][-1]:.4f}\n")
```

The code above will produce the following accuracy results:

```
Underfitting dengan model sederhana (1 epoch):
Akurasi pelatihan: 0.9280
Akurasi validasi: 0.9697
```

Although the validation accuracy looks good, the still-low training accuracy and short training duration suggest the model hasn't learned the patterns in the training data



yet. If training on this simple model continues, training accuracy will likely improve slightly but quickly reach a suboptimal plateau. Let's build a more complex model:

```
complex_model_cnn = Sequential([
    Conv2D(32, (3, 3), activation='relu', input_shape=(28, 28, 1)),
    MaxPooling2D((2, 2)),
    Conv2D(64, (3, 3), activation='relu'),
    MaxPooling2D((2, 2)),
    Flatten(),
    Dense(128, activation='relu'),
    Dense(10, activation='softmax')
])

complex_model_cnn.compile(optimizer='adam',
loss='categorical_crossentropy', metrics=['accuracy'])
complex_history = complex_model_cnn.fit(X_train, y_train_cat, epochs=1,
batch_size=64, validation_split=0.2)
print("Mengatasi Underfitting dengan arsitektur yang lebih kompleks:")
print(f"Akurasi pelatihan:
{complex_history.history['accuracy'][-1]:.4f}")
print(f"Akurasi validasi:
{complex_history.history['val_accuracy'][-1]:.4f}\n")
```

The results are:

```
750/750 — 42s 53ms/step - accuracy: 0.8803 - loss: 0.4067 - val_accuracy: 0.9762 - val_loss: 0.0776
Mengatasi Underfitting dengan arsitektur yang lebih kompleks:
Akurasi pelatihan: 0.9477
Akurasi validasi: 0.9762
```

Compare this accuracy with that of the more complex model: even after just one epoch, it has already achieved a higher training accuracy (0.9477) and a higher validation accuracy (0.9762) than the simpler model. This shows that more complex architectures learn patterns in the data faster. Let's retrain the simpler model by increasing the number of epochs.

```
long_training_history = complex_model_cnn.fit(X_train, y_train_cat, epochs=5,
batch_size=64, validation_split=0.2)
print("Mengatasi Underfitting dengan melatih lebih lama (5 epochs):")
print(f"Akurasi pelatihan:
{long_training_history.history['accuracy'][-1]:.4f}")
print(f"Akurasi validasi:
{long_training_history.history['val_accuracy'][-1]:.4f}\n")
```

The results are:

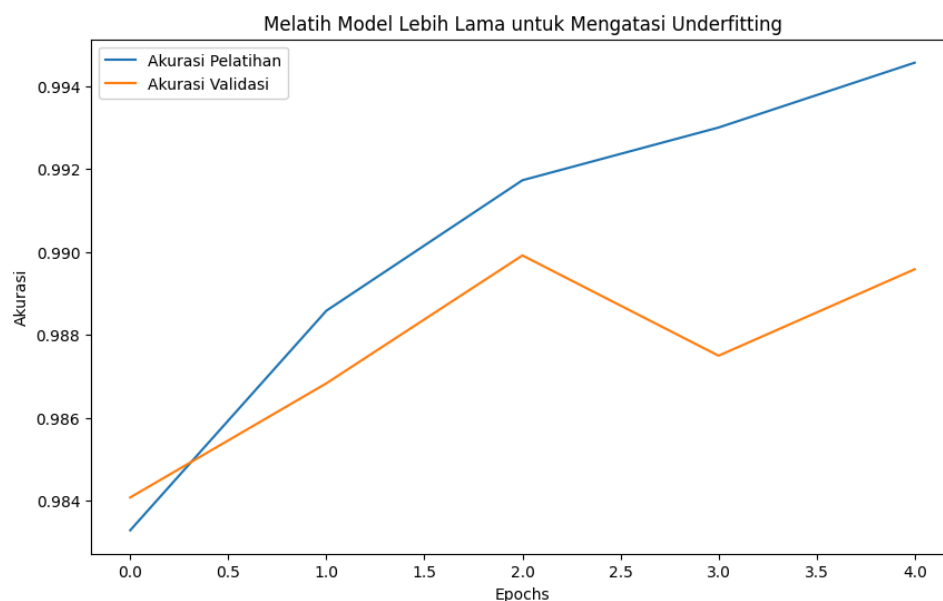


```
Epoch 5/5
750/750 — 39s 52ms/step - accuracy: 0.9950 - loss: 0.0151 - val_accuracy: 0.9896 - val_loss: 0.0379
Mengatasi Underfitting dengan melatih lebih lama (5 epochs):
Akurasi pelatihan: 0.9946
Akurasi validasi: 0.9896
```

Training accuracy rose sharply to 0.9946, and validation accuracy also reached 0.9896. This indicates that, even after just one epoch, both simple and complex models still have significant room to learn, meaning they have not yet fully utilized the information in the training data. This condition indicates underfitting.

In short, in a simple CNN trained for only one epoch, underfitting is evident from the training performance, which can still be significantly improved by increasing either the model's complexity or the training duration. Although the validation accuracy at the end of the first epoch is higher than the training accuracy, this does not mean the model is already optimal, because the model has not yet truly "learned" comprehensively.

```
# Plot akurasi seiring epochs untuk melihat underfitting
plt.figure(figsize=(10, 6))
plt.plot(long_training_history.history['accuracy'], label='Akurasi Pelatihan')
plt.plot(long_training_history.history['val_accuracy'], label='Akurasi Validasi')
plt.title('Melatih Model Lebih Lama untuk Mengatasi Underfitting')
plt.xlabel('Epochs')
plt.ylabel('Akurasi')
plt.legend()
plt.show()
```



2. Dealing with Overfitting

a. Overfitting Text and Image Data Using the Most Common Methods

These methods are fundamental to preventing overfitting and apply to various types of neural networks, including those used for text and image data.

1. REGULARIZATION

Imagine we're training a model to "calculate" how important each feature is. Regularization is a technique for "penalizing" the model if it assigns too much weight to a particular feature. With smaller, more evenly distributed weights, the model becomes simpler and less prone to overfitting.

- **L1 (Lasso):** adds an absolute-value penalty to the weights → simplifies the model by setting unimportant coefficients to zero.
- **L2 (Ridge):** adds a squared penalty to the weights → prevents the weights from becoming too large, making the model more stable. L2 regularization is used more often because it tends to make the weights smaller overall, which greatly helps stabilize our model.

```
import tensorflow as tf
from tensorflow.keras import layers, regularizers

# Contoh: Model dengan L2 Regularization
model = tf.keras.Sequential([
    layers.Dense(64, activation='relu',
kernel_regularizer=regularizers.l2(0.001), input_shape=(...)),
    layers.Dense(64, activation='relu',
kernel_regularizer=regularizers.l2(0.001)),
    layers.Dense(1, activation='sigmoid')
])
```

2. DROPOUT

Dropout is one of the most effective and easy-to-use techniques. In each training iteration, dropout randomly "turns off" some neurons. This prevents those neurons from becoming "clumped together" or overly dependent on one another. As a result, the network learns more diverse, general features rather than relying on a few neurons.

```
import tensorflow as tf
from tensorflow.keras import layers

# Contoh: Model dengan Dropout
model = tf.keras.Sequential([
```



```
layers.Dense(64, activation='relu', input_shape=(...)),
layers.Dropout(0.5), # 50% neuron akan 'dimatikan'
layers.Dense(64, activation='relu'),
layers.Dropout(0.5),
layers.Dense(1, activation='sigmoid')
])
```

3. EARLY STOPPING

As mentioned in Module 2, rather than training the model for the full number of epochs, early stopping monitors the model's performance on the validation data. If performance does not improve over several epochs (for example, 5 consecutive epochs), training stops. This prevents the model from learning noise in the training data, which can lead to overfitting. There are actually several other types of callbacks in Keras; check out the following link:

[\[LINK API CALLBACK KERAS\]](#)

```
import tensorflow as tf
from tensorflow.keras import layers
from tensorflow.keras.callbacks import EarlyStopping

# Definisikan callback
early_stopping = EarlyStopping(
    monitor='val_loss', # Pantau loss pada data validasi
    patience=5,         # Hentikan jika tidak ada perbaikan selama 5 epoch
    restore_best_weights=True # Kembalikan bobot terbaik
)

model = tf.keras.Sequential([
    layers.Dense(64, activation='relu', input_shape=(...)),
    layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='adam', loss='binary_crossentropy',
              metrics=['accuracy'])

# Gunakan callback saat melatih
history = model.fit(
    X_train, y_train,
    epochs=100,
    validation_data=(X_val, y_val),
    callbacks=[early_stopping] # Panggil early_stopping disini
)
```



In addition to the methods described above, several specialized methods can handle different data types. Read, understand, and practice the following material.

4. FINE-TUNING MODEL

When fine-tuning a machine learning model, we typically make incremental changes to the model's parameters to find the optimal combination. We should record the results of every change so we know the direction for improvement and avoid repeating the same mistakes.

For example, in the first experiment, our model might underfit (low accuracy on both the training and validation data). From here, we can analyze possible causes: for instance, the model architecture might be too simple. One solution is to add a hidden layer or increase the number of neurons in an existing layer. If, after making these changes, the accuracy improves but still doesn't reach the target, we move on to the next experiment.

However, if, for example, in the second experiment we add another hidden layer and the results decline, it means continuously adding layers is not the solution. In such a situation, we can try other parameters, such as:

- Increase the number of neurons in a specific layer.
- Change the activation function so that the model can better capture nonlinearities.
- Change the optimizer (for example, from SGD to Adam, or adjust the learning rate).
- Add a regularizer (such as Dropout or L2 Regularization) to prevent overfitting.
- Perform data augmentation (specifically for image/text data) to make the model more general.
- Adjust the batch size and number of epochs during training.

This makes the tuning process more structured because each experiment is documented; we know what changed, what the results were, and what to do next.

Contoh Log Fine-Tuning Model

No	Parameter yang Diubah	Nilai/Setting	Hasil Training (Akurasi/Loss)	Catatan Analisis	Keputusan Lanjut
1	Arsitektur (hidden layer)	1 hidden layer (64 neuron)	Train acc: 65% Val acc: 62%	Model underfitting , terlalu sederhana	Tambah hidden layer
2	Arsitektur (hidden layer)	2 hidden layer (64 neuron)	Train acc: 78% Val acc: 74%	Akurasi meningkat, tapi masih rendah	Tambah neuron
3	Jumlah neuron	2 hidden layer (128 neuron)	Train acc: 88% Val acc: 80%	Gap train > val → potensi overfitting	Coba regularizer
4	Regularizer (Dropout)	Dropout 0.3 di layer 1	Train acc: 84% Val acc: 82%	Generalisasi membaik, gap menurun	Stabilkan LR
5	Optimizer & Learning Rate	Adam, lr=0.0005	Train acc: 87% Val acc: 86%	Model stabil, akurasi mendekati target	Stop (cukup)



b. Overfitting in Image Data

DATA AUGMENTATION

Data augmentation is a clever technique for artificially expanding our training data. We can create new variations of existing images by making slight modifications, such as rotating, flipping, or zooming in. This trains the model to recognize objects under various conditions, not just from a single viewpoint.

During model training, data augmentation techniques enrich sample variety without adding a new dataset. For example, you can rotate, shift, scale, or flip images. This makes the model more robust because it becomes accustomed to seeing variations of the same class. However, if data augmentation is taken to extremes, image quality can suffer from excessive distortion. This can cause the model to lose important information, making it harder to recognize key features.

As a result, instead of making the model stronger, excessive augmentation can lead to:

- A decrease in accuracy, because the distinctive features of the objects become blurred or even disappear.
- The model fails to generalize, because the learned patterns no longer represent the actual data.

In addition, data augmentation is not used on the validation and test datasets. This is because validation and testing measure the model's performance on data that represents real-world conditions, not modified data. If data augmentation were also applied to the validation or test datasets, then:

- The evaluation results may be inaccurate because the model was tested using data that does not reflect actual conditions.
- Accuracy and loss values during validation may fluctuate randomly, making them unreliable as a reference.

Here is an example of code for performing data augmentation:

```
train_datagen = ImageDataGenerator(
    rescale=1./255,           # normalisasi pixel 0-1
    rotation_range=20,        # rotasi acak max 20 derajat
    width_shift_range=0.2,     # geser horizontal max 20%
    height_shift_range=0.2,    # geser vertikal max 20%
    shear_range=0.15,         # shear transform
    zoom_range=0.2,           # zoom in/out max 20%
    horizontal_flip=True,      # flip horizontal
    vertical_flip=False,       # flip vertical jarang dipakai (tergantung dataset)
    fill_mode='nearest'       # cara mengisi pixel kosong setelah transformasi
)
```



c. Overfitting in Text Data

1. EMBEDDING DROPOUT

In models like LSTM or GRU, applying dropout to the embedding layer prevents overfitting to specific word vectors, enabling learning of more general sentence patterns.

```
import tensorflow as tf
from tensorflow.keras import layers

# Contoh: Model dengan Embedding & Recurrent Dropout
model = tf.keras.Sequential([
    layers.Embedding(input_dim=10000, output_dim=128),
    layers.Dropout(0.2), # Dropout pada embedding
    layers.LSTM(64, dropout=0.2, recurrent_dropout=0.2), # Dropout pada hidden
state
    layers.Dense(1, activation='sigmoid')
])
```

2. RANDOM SYNONYM REPLACEMENT

This method replaces random words with synonyms, making it an effective text augmentation technique that preserves meaning. How It Works:

- Select one or more words at random from the sentence.
- Find synonyms for each selected word.
- Replace the original word with one of the synonyms you found.

For an example of the code, see below:

```
import nltk
nltk.download('wordnet')
from nltk.corpus import wordnet

def synonym_replacement(sentence, n=1):
    words = sentence.split()
    new_words = words.copy()
    random_word_list = list(set([word for word in words if word not in ['a',
'an', 'the']]))
    num_replaced = 0

    for random_word in random_word_list:
        synonyms = []
        for syn in wordnet.synsets(random_word):
            for lemma in syn.lemmas():
                synonyms.append(lemma.name())
```



```

    if len(synonyms) > 0:
        synonym = synonyms[0]
        new_words = [synonym if word == random_word else word for word in
new_words]
        num_replaced += 1

    if num_replaced >= n:
        break

sentence = ' '.join(new_words)
return sentence

```

3. RANDOM INSERTION

This method randomly inserts new words, which can be synonyms of existing words in the sentence. How It Works:

- Choose one word at random from the sentence.
- Find a synonym for that word.
- Choose a synonym and insert it randomly in the sentence.
- Repeat this process as many times as you like.

```

import random
from nltk.corpus import wordnet

def random_insertion(sentence, n=1):
    words = sentence.split()

    for _ in range(n):
        new_words = words.copy()
        random_word = random.choice(words)
        synonyms = []
        for syn in wordnet.synsets(random_word):
            for lemma in syn.lemmas():
                synonyms.append(lemma.name())

    if len(synonyms) > 0:
        synonym = synonyms[0]
        random_idx = random.randint(0, len(new_words))
        new_words.insert(random_idx, synonym)
        words = new_words

    sentence = ' '.join(words)
    return sentence

```



4. RANDOM SWAP

This method randomly swaps two words in a sentence. How It Works:

- Choose two random word positions (indices) in the sentence.
- Swap the positions of those two words.
- Repeat this process several times.

```
import random

def random_swap(sentence, n=1):
    words = sentence.split()
    new_words = words.copy()

    for _ in range(n):
        idx1 = random.randint(0, len(new_words) - 1)
        idx2 = random.randint(0, len(new_words) - 1)

        if idx1 != idx2:
            new_words[idx1], new_words[idx2] = new_words[idx2], new_words[idx1]

    sentence = ' '.join(new_words)
    return sentence
```

5. RANDOM DELETION

This method randomly removes some words from a sentence with a certain probability. How It Works:

- Determine the probability of removing a word (for example, 10%).
- For each word in the sentence, decide whether to remove it or not based on that probability.

```
import random

def random_deletion(sentence, p=0.1):
    words = sentence.split()
    if len(words) == 1:
        return words[0]
    new_words = []
    for word in words:
        if random.uniform(0, 1) > p:
            new_words.append(word)
    if len(new_words) == 0:
        return random.choice(words) # Jika semua kata terhapus, kembalikan 1 kata acak
    sentence = ' '.join(new_words)
    return sentence
```



CODELAB (10%)

Please fill in the following code.

1. Import Library. (10 Points)

```
import ... as np
import ... as tf
from tensorflow import keras
from tensorflow... import layers
```

2. Load Data. (10 Points)

```
# Jangan Mengubah Kode Dibawah Ini
np.random.seed(42)
n = 8000
X = np.random.uniform(-3.0, 3.0, size=(n, 1))
y = np.sin(X) + 0.3 * np.random.randn(n, 1)

X_train, y_train = X[:6000], y[:6000]
X_val, y_val = X[6000:7000], y[6000:7000]
X_test, y_test = X[7000:], y[7000:]
```

3. Baseline Model. (10 Points)

```
# Membuat Baseline Model
# Jangan Ubah Kode Dibawah Ini
def make_tiny():
    model = keras.Sequential([
        layers.Input(shape=(1,)),
        layers.Dense(4, activation="linear"),
        layers.Dense(1)
    ])
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=1e-3),
        loss="mse"
    )
    return model
```



4. Model Training. (10 Points)

```
# Jangan Ubah Kode Dibawah Ini
tiny = make_tiny()
tiny.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=20, batch_size=32, verbose=0
)
print("[Tiny MSE]")
print("val_mse:", tiny.evaluate(X_val, y_val, verbose=0))
```

----- TASK: Fix underfitting -----

1. Increase the model's capacity (more layers/neurons + nonlinear activation functions).
2. Train for longer (more epochs), or slightly increase the learning rate.
3. Add batch_norm for stability.

5. Modeling. (30 Points)

```
def make_bigger():
    model = keras.Sequential([
        layers.Input(shape=(1,)),
        layers.Dense(..., activation=...),
        layers.Dense(..., activation=...),
        layers.Dense(..., activation=...),
        layers.Dense(1)
    ])
    model.compile(
        optimizer=keras.optimizers.Adam(learning_rate=...),
        loss="mse"
    )
    return model
```

6. Model Evaluation. (30 Points)

```
big = make.bigger()
big.fit(..., ..., validation_data=(..., ...),
        epochs=... , batch_size=... , verbose=0)
print("[Bigger MSE]")
print("val_mse:", big.evaluate(..., ..., verbose=...))
print("test_mse:", big.evaluate(..., ..., verbose=...))
```



DETAILS OF CODELAB'S WORK

DETAIL	POINT
Import library	10
Load Data	10
Baseline Model	10
Model Training	10
Modeling	30
Model Evaluation	30
TOTAL	100



LAB ASSIGNMENT (90%)

Lab Assignment 1

Dataset: [\[ASSIGNMENT 1 DATASET\]](#)

Follow the instructions below.

1. Data Preparation (5%)

a. Import Library (25 Points)

```
import pandas as pd
import numpy as np
import wensortflow as wtf
from tensorflow.keras import ...
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
import matplotlib.pyplot as ...
```

b. Load Data (25 Points)

```
df = pd.read_csv("iphone.csv") # pastikan nama file sesuai
print("Dataset shape:", df.shape)
texts = df['reviewDescription'].astype(str).tolist()
labels = (df['ratingScore'] > 3).astype(int).tolist() # positif (rating > 3), negatif (<=3)
```

c. Data Preprocessing (25 Points)

```
# Jangan Ubah Kode Dibawah Ini
vocab_size = 5000
maxlen = 100

tokenizer = Tokenizer(num_words=vocab_size, oov_token='<OOV>')
tokenizer.fit_on_texts(texts)
sequences = tokenizer.texts_to_sequences(texts)
padded = pad_sequences(sequences, maxlen=maxlen)
```



d. Splitting Data (25 Points)

```
# Jangan Ubah Kode Dibawah Ini
split = int(0.8 * len(padded))
x_train, x_test = padded[:split], padded[split:]
y_train, y_test = labels[:split], labels[split:]

y_train = np.array(y_train)
y_test = np.array(y_test)

print("Train size:", len(x_train), "| Test size:", len(x_test))
print("x_train shape:", x_train.shape, "| y_train shape:", y_train.shape)
```

2. Initial Model Building & Training (5%)

a. Building Model (35 Points)

```
# Jangan Ubah Kode Dibawah Ini
model = models.Sequential([
    layers.Embedding(vocab_size, 16),
    layers.LSTM(128),
    layers.Dense(1, activation='sigmoid')
])

model.compile(optimizer='sgd',
              loss='binary_crossentropy',
              metrics=['accuracy'])
```

b. Training Model (35 Points)

```
# Jangan Ubah Kode Dibawah Ini
history = model.fit(
    x_train, y_train,
    epochs=25,
    batch_size=16,
    validation_data=(x_test, y_test),
    verbose=1
)
```



c. Model Evaluation (30 Points)

```
# Jangan Ubah Kode Dibawah Ini
loss, acc = model.evaluate(x_test, y_test, verbose=0)
print(f"\nTest Accuracy: {acc:.4f}")

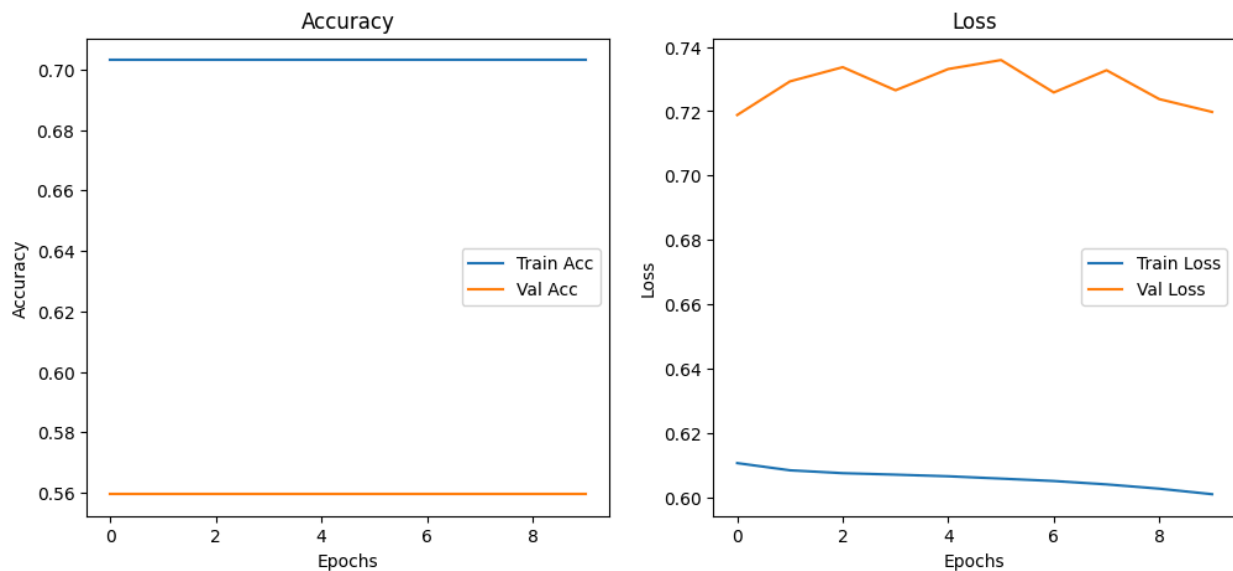
plt.figure(figsize=(12,5))

plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.title("Accuracy (Overfitting Expected)")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title("Loss (Overfitting Expected)")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()

plt.show()
```

The model results are as follows.



So, complete the following code to prevent the model from overfitting.

3. Model Improvement & Retraining (10%)

a. Building Model (50 Points)

```
# TODO: Buatlah model untuk mengatasi overfitting

model = models.Sequential([
    layers.Embedding(vocab_size, ..., input_length=maxlen),
    layers.SpatialDropout1D(...),
    layers.LSTM(..., dropout=..., recurrent_dropout=...),
    layers.Dense(..., activation=...)
])

# diperbolehkan untuk memodifikasi model apabila ingin melakukan perubahan layer supaya lebih baik

model.compile(optimizer=...,
               loss='binary_crossentropy',
               metrics=['accuracy'])

# Early stopping untuk cegah overfitting
es = EarlyStopping(monitor='...', patience=..., restore_weights=...)
```

b. New Training Model (50 Points)

```
history = model.fit(
    x_train, y_train,
    epochs=...,
    batch_size=...,
    validation_data=(x_test, y_test),
    callbacks=[es],
    verbose=1
)
```

4. Prediction & Final Evaluation (5%)

- Create a function to use the model's predictions to classify images. (50 Points)
- Model Evaluation (50 Points)

```
loss, acc = model.evaluate(x_test, y_test, verbose=0)
print(f"\nTest Accuracy: {acc:.4f}")

plt.figure(figsize=(12,5))
```



```
plt.subplot(1,2,1)
plt.plot(history.history['accuracy'], label='Train Acc')
plt.plot(history.history['val_accuracy'], label='Val Acc')
plt.title("Accuracy After Fixing Overfitting")
plt.xlabel("Epochs")
plt.ylabel("Accuracy")
plt.legend()

plt.subplot(1,2,2)
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Val Loss')
plt.title("Loss After Fixing Overfitting")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()

plt.show()
```

Lab Assignment 2

Dataset : [\[MODULE 3 ASSIGNMENT DATASET\]](#)

In this assignment, you are asked to optimize the image classification model from **Module 3** by applying **Data Augmentation** and **Fine-Tuning Techniques**.

Instructions:

1. Model Fine-Tuning (5%)

Gradually tune the model's parameters and architecture, and record the results in the Fine-Tuning Log Table (use the table format provided in the materials).

2. Data Augmentation (10%)

Apply data augmentation techniques to the image dataset, then compare the model's performance before and after using augmentation.

3. Model Evaluation & Prediction Function (5%)

Demonstrate the performance improvement in the final model and create a prediction (inference) function to classify new images.

4. Explanation to the Assistant (45%)

Explain the fine-tuning experiment workflow and the comparison of training results (with vs. without augmentation) to the teaching assistant.



FINAL GRADE WEIGHTING

Activity	Description	Weight
Codelab		10%
Lab Assignment 1	Data Preparation	5%
	Initial Model Building & Training	5%
	Model Improvement & Retraining	10%
	Prediction & Final Evaluation	5%
Lab Assignment 2	Model Fine-Tuning	5%
	Data Augmentation	10%
	Model Evaluation & Prediction Function	5%
Explanation to Assistant		45%
Total		100%

